

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	RAGUL.M	Reg. No.:	192524066
Section / Batch:	SLOT-A	Date:	26/08/2026

Code of Conduct

I RAGUL.M / 192524066

(Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ragul

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

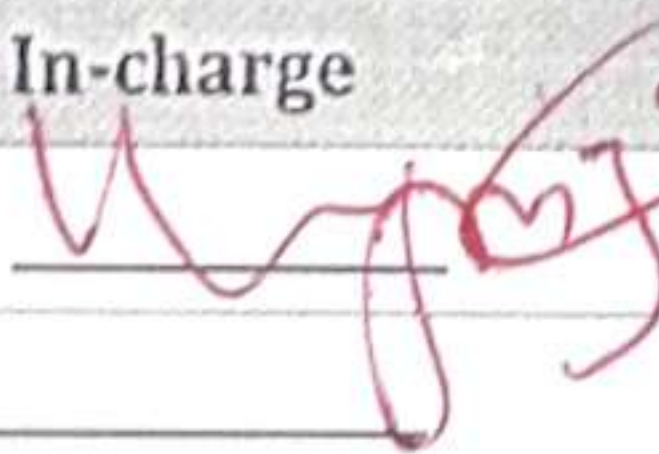
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on:
(20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays?
(20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of:
(20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following:
(20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

compare linear probing and separate chaining for collision handling in hashing based on.

Q1 clustering effect:

→ Linear probing: suffers heavily from primary clustering when collision occur. contiguous blocks of occupied slots form, causing search, inserting, and deletion operations to degrade as new keys land within as near these clusters

→ Separate chaining: completely eliminates primary and secondary clustering collisions are handled in independent linked lists (or dynamic data structure) at each bucket index, so filled buckets do not block adjacent array slots.

Q2 memory usage:

→ Linear probing: Extremely space-efficient. It stores all key-value pairs directly in the table array without requiring extra node pointers or dynamic allocation, maximizing CPU cache locality

→ Separate chaining: incurs memory overhead due to pointers storage (e.g. next pointers in linked lists for every key). Additional memory fragmentation can also occur due to dynamic node allocation outside contiguous memory.

c] Performance under High Load Factor ($\alpha = n/m$)

→ Linear probing: performance degrades rapidly as the load factor approaches or exceeds 0.75. Search and insertion times increase significantly due to long probing sequence, eventually failing if $\alpha > 1.0$ without resizing.

→ Separate chaining gracefully handles high load factors. Search time grows linearly with the average chain length $(1 + \alpha)$ but operations never fail due to running out of table slots.

d] Suitability for Large-Scale Applications

Recommendation: Separate chaining (or hybrid variants using balanced trees for dynamic chains) is more suitable for large scale unpredictable application.

Reasoning: Large scale system frequently handle dynamic dataset growth where key counts fluctuate significantly. Separate chaining growth where key counts fluctuates, catastrophic degradation or service interruption when the load factor spikes handled. dynamic resizing smoothly, and avoids the performance bottle necks associated with some clustering under heavy loads, linear probing is preferred. Only when loads linear probing is preferred only when table size is can be pre-allocated accurately and maximum cache performance is required.

2 Comparative Analysis: Quick Sort vs merge Sort for 10 million linked list nodes.

i] memory Requirements (in-place vs Extra Space).

→ merge Sort: Extremely Space efficient for linked lists operating in $O(\log n)$ extra space for recursion stacks (or $O(1)$ space iteratively). Recombining is accomplished entirely by manipulating pointers without allocating any memory for data items.

→ Quick Sort: Requires $O(\log n)$ auxiliary space on average (up to $O(n)$ in the worst case) purely for the call stack; while it does not allocate any copies of stack space for 10 million elements under bad pivots can cause significant memory pressure.

ii] Cache Behaviour

merge Sort: poor cache locality since nodes of a large 10-million node list are scattered across heap memory traversing them during split and merge steps leads to frequent CPU cache misses.

Quick Sort: Also exhibits poor cache locality on linked lists due to non-contiguous node memory addresses during partitioning.

iii] Time complexity

merge Sort: consistently $O(n \log n)$ across best, average and worst cases. Having linked lists takes $O(n)$ time by using fast and slow pointers, making total runtime predictable.

Quick Sort: Average time complexity is $O(n \log n)$, but worst case time complexity degrades to $O(n^2)$ if poor pivots (eg: sorted data) are picked partitioning linked lists without random access to pick a good pivot adds overhead.

iv] Preferred Algorithm & Comparative Justification

→ FOR Linked lists: Merge Sort is strongly preferred.

→ why: Finding the middle of a linked list and merging sublists only requires pointer manipulation without random element access. merging takes $O(1)$ extra structural space and Merge Sort guarantees $O(n \log n)$ worst case performance without stack overflow risks.

→ How the Answer changes for Arrays:

Quick Sort becomes the preferred choice why: Arrays. Support random access ($O(1)$ index lookup), allowing Quick Sort to pick medium pivots easily and partition elements in place. With sequential memory access this gives Quick Sort superior CPU cache locality and lower hidden constant factors compared to merge sort, which requires $O(n)$ extra memory allocation for array copies.

3. comparative Analysis: Quadratic probing vs double Hashing.

a) clustering issues:

Eliminates primary clustering by probing using a quadratic polynomial $(h(k_i) = (h'(k) + (i^2 + c_2 i)) \pmod{m})$

However it suffers from secondary clustering. Keys that hash to the same initial index follow the exact same probe sequence.

Double hashing:

Eliminates both primary and secondary clustering. It uses a secondary hash function to calculate dynamic step sizes.

$(h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod{m})$, ensuring that two keys hashing to the same initial follow entirely different probe sequence based on $h_2(k)$

b) collision resolution efficiency:

Quadratic probing: Highly efficient for moderate table loads. It minimizes memory access latency by probing closer indices compared to double hashing but efficiency degrades if secondary clustering causes long probe sequences.

Double hashing : offers the highest collision resolution efficiency among open addressing techniques its uniform distribution of probe sequence minimizes probe lengths even under higher load factors. though computing two separate hash junction introduces minor extra computational overhead per collision

c. Suitability for Dense Hash tables:

→ Quadratic probing : less suitable for dense tables probe sequences may fail to explore all table slots unless the table size is prime and the load factor is kept strictly below $\alpha \leq 0.5$.

→ Double Hashing : more suitable for dense hash tables because $h_2(k)$ and table size m can be chosen to be relatively prime (ensuring a permutation of all table slots). Double Hashing successfully searches through more available slots and maintains better efficiency at higher load factors before requiring a table resize.

4 Comparison of Best Case and Worst Case time Complexity of Sorting algorithms.

a) practical Significance:

Best Case time Complexity:

Definition Represents the minimum time an algorithm needs to execute, occurring under ideal input condition (eg. an array that is already fully sorted).

practical Role:

useful for identifying algorithms optimized for pre-sorted, or nearly-sorted data for instance. insertion sort and bubble sort run in $O(n)$. best-case time when an array is already sorted. making them highly efficient for nearly sorted inputs or online data streams.

Worst Case Time Complexity

Definition: Represents the maximum time an algorithm requires to run occurring under the most unfavorable input arrangements.

practical Role: Serves as a performance guarantee real-world system. Involves critical applications and real-time environments on worst case bounds to prevent unexpected performance degradation application freezes, or denial of service (DOS).

b. Impact Algorithm Selection:

Guarantee vs optimism:

→ Engineers prioritize worst case complexity when stability and guaranteed response times are required.

→ Algorithms like merge sort and heap sort offer a predictable $O(n \log n)$ worst case bound making them reliable choices for general purpose libraries and real time systems.

→ Context Aware Algorithm choices.

Quick Sort: Has a best average case time

Complexity of $O(n \log n)$ but a worst case of $O(n^2)$ (occurring with poor pivot selection on sorted arrays). Despite this randomized or dual-pivot quick sort is preferred in practice for cache locality and low constant factors. Provided worst case mitigation (like introsort) is implemented.

Algorithm	Best Case Time	Worst Case Time
Insertion Sort	$O(n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$

5. Comparison of recursive and iterative implementations of Quick Sort

Space Complexity (Stack usage).

Recursive implementation

→ Uses the implicit system call stack to store activation records (function parameters, local variable return addresses) for each recursive call.

→ Worst case space: $O(n)$ call stack depth if the pivot consistently yields completely unbalanced partitions.
(e.g.) Sorted input with first/last element chosen as pivots

Iterative implementation

→ Replaces implicit system recursion with an explicit user defined stack data structure (e.g., storing range pairs, $[low, high]$)

→ Worst case space: $O(n)$ explicit stack allocation if partitions are completely unbalanced

→ Best/Average case space: $O(\log n)$ space when processing the smaller partitions first before pushing the larger range onto the explicit stack.

b. Fare of implementation:

Recursive implementation:

High Readability: extremely concise, elegant, and straight forward to write. The algorithm clearly matches the divide and conquer strategy requiring only a few lines to partition and invoke recursive calls on subarrays.

Iterative implementation

Higher complexity: Requires manually managing a custom stack array (structure), pushing and popping index pairs (low and high) and implementing explicit loop control logic.

c. Risk of Stack Overflow

Recursive implementation:

High Risk: Deep recursion on large dataset ($n \geq 10^5$ or 10^6) with unbalanced pivots quickly exceeds the limited system stack size (typically 1MB to 8MB) resulting in a runtime stack overflow error on crash.

Iterative implementation:

Low/No System Stack Risk: Allocates stack space dynamically on the heap rather than relying on call stack execution limits. Heap memory is vastly larger than system stack space, effectively eliminating the risk of stack